

Homework 4: Neural Networks

This assignment is due on Gradescope by **11:59pm on Friday, May 22nd**. Your solutions to theoretical questions should be done in Markdown/Latex directly below the associated question. Your solutions to computational questions should include any specified Python code and results as well as written commentary on your conclusions. Remember that you are encouraged to discuss the problems with your instructors and classmates, but **you must write all code and solutions on your own**.

NOTES:

- Some problems with code may be autograded. If we provide a function API **do not** change it. If we do not provide a function API then you're free to structure your code however you like.
- Submit this Jupyter notebook and a PDF of your output to Canvas. Do not compress it using tar, rar, zip, etc.
- In this homework, we will use [PyTorch](#) to implement a classifier. Install the current stable release for PyTorch (instructions on homepage of Pytorch website). We recommend you use Anaconda for these installations in a virtual environment but you can use pip as well. For this homework, this command should be sufficient `pip3 install torch`.

It is **highly recommended** you install the CPU-only version of PyTorch if you're unfamiliar with the process of installing CUDA libraries for these packages. If you choose to install a GPU version, ensure your code runs without GPU support.

Put your name here:

Sikhan

```
In [14]: # Test PyTorch installation
# Please do not edit this cell
import torch
import math
import pickle
import gzip
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

torch.manual_seed(1234)
```

```
Out[14]: <torch._C.Generator at 0x11ca55d10>
```

```
In [15]: # Put your imports (if any) here
```

[26 points] Problem 1 - Single-Layer and Multilayer Perceptron Learning

Part 1 [13 points]

In general, a *single-layer perceptron* is a neural network with an input layer, no hidden layers, and an output layer. *Multilayer perceptrons* can have one or more hidden layers.

Consider learning the following concepts with either a single-layer or multilayer perceptron where all hidden and output neurons utilize the *indicator* activation functions. For each of the following concepts, state whether the concept can be learned by a single-layer perceptron.

Briefly justify your response by providing weights, biases, and the indicator activation functions if applicable:

- **NOT** x_1
- x_1 **NAND** x_2
- x_1 **XNOR** x_2 (output 1 when $x_1 = x_2$ and 0 otherwise)

All neurons use the indicator activation $1[z \geq 0]$.

- **NOT** x_1 — **yes**. With one input it is trivially separable: weight -1 and bias 0.5 give 1 when $x_1 = 0$ and 0 when $x_1 = 1$.
- x_1 **NAND** x_2 — **yes**. Only $(1, 1)$ outputs 0, and one line separates that corner from the rest. Weights $(-1, -1)$, bias 1.5 .
- x_1 **XNOR** x_2 — **no**. The positives $(0, 0)$, $(1, 1)$ and the negatives $(0, 1)$, $(1, 0)$ sit on the two diagonals of the unit square, so no single line separates them (the same reason XOR fails). A hidden layer is needed, see Part 2.

Part 2 [13 points]

Determine an architecture and specific values of the weights and biases in a single-layer or multilayer perceptron with indicator activation functions that can learn x_1 **XNOR** x_2 . Make a truth table of x_1 , x_2 , and x_1 **XNOR** x_2 , describe your perceptron's architecture, and state your weight matrices and bias vectors in Markdown below. Then demonstrate that your solution is correct by implementing forward propagation for your network in Python and showing that it produces the correct boolean output values for each of the four possible combinations of x_1 and x_2 .

Use two layers: the hidden layer computes AND and NOR of the inputs, and the output ORs them — XNOR is true exactly when one of those two fires.

x_1	x_2	XNOR
0	0	1
0	1	0
1	0	0
1	1	1

Architecture: 2 inputs $\rightarrow$ 2 hidden units $\rightarrow$ 1 output, all using the indicator activation. Hidden unit 1 is AND, hidden unit 2 is NOR, and the output is OR.

$$W^1 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad b^1 = \begin{bmatrix} -1.5 \\ 0.5 \end{bmatrix}, \quad W^2 = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b^2 = \begin{bmatrix} -0.5 \end{bmatrix}$$

The cell below runs forward propagation on all four inputs and reproduces the XNOR column.

```
In [16]: def indicator(z):
# step activation: 1 if z >= 0 else 0
return (z >= 0).astype(float)

W1 = np.array([[1.0, 1.0],
               [-1.0, -1.0]])
b1 = np.array([[-1.5],
               [0.5]])
W2 = np.array([[1.0, 1.0]])
b2 = np.array([[ -0.5]])

print("x1 x2 | XNOR")
for x1 in [0, 1]:
    for x2 in [0, 1]:
        x = np.array([[x1], [x2]], dtype=float)
        h = indicator(W1 @ x + b1)
        out = indicator(W2 @ h + b2)
        print(f" {x1} {x2} | {int(out[0, 0])}")
```

```
x1 x2 | XNOR
0 0 | 1
0 1 | 0
1 0 | 0
1 1 | 1
```

[30 points] Problem 2 - Backpropagation

In this problem you will gain some intuition about why training deep neural networks can be very time consuming. Consider training a chain-like neural network:

 chain-like nn

Note that this network has three weights W^1, W^2, W^3 and three biases b^1, b^2 , and b^3 (for this problem you can think of each parameter as a single value or as a 1×1 matrix). Suppose that each hidden and output neuron is equipped with a sigmoid activation function and the loss function is given by

$$\ell(y, a^3) = \frac{1}{2}(y - a^3)^2$$

where a^3 is the value of the activation at the output neuron and $y \in \{0, 1\}$ is the true label associated with the training example.

Note: Superscripts on W 's, b 's, and a 's are being used as indices and not powers here (e.g. a^3 indicates the 3rd activation value, not the activation value a to the 3rd power.)

Part 1 [5 points]

Suppose each of the weights is initialized to $W^k = 1.0$ and each bias is initialized to $b^k = -0.5$. Use forward propagation to find the activities and activations associated with each hidden and output neuron for the training example $(x, y) = (0.5, 0)$. Show your work.

Each layer computes $z^k = W^k a^{k-1} + b^k$ and then $a^k = \sigma(z^k)$, starting from $a^0 = x = 0.5$. Since $1 \cdot 0.5 - 0.5 = 0$ at every layer, all three activities are $z^k = 0$ and all three activations are $a^k = \sigma(0) = 0.5$.

The loss is $\ell = \frac{1}{2}(y - a^3)^2 = \frac{1}{2}(0 - 0.5)^2 = 0.125$.

Part 2 [8 points]:

Use backpropagation to compute the weight and bias derivatives $\partial\ell/\partial W^k$ and $\partial\ell/\partial b^k$ for $k = 1, 2, 3$. Show all work.

Let $\delta^k = \partial\ell/\partial z^k$. Here every neuron has $\sigma'(z^k) = a^k(1 - a^k) = 0.25$, and $\partial\ell/\partial a^3 = a^3 - y = 0.5$. Going backward, each step just multiplies by the next weight ($= 1$) and by 0.25:

$$\delta^3 = 0.125, \quad \delta^2 = 0.03125, \quad \delta^1 = 0.0078125$$

Then $\partial\ell/\partial W^k = \delta^k a^{k-1}$ and $\partial\ell/\partial b^k = \delta^k$ (with $a^0 = x = 0.5$):

k	$\partial\ell/\partial W^k$	$\partial\ell/\partial b^k$
3	0.0625	0.125
2	0.015625	0.03125
1	0.00390625	0.0078125

Each δ shrinks by a factor of 0.25 per layer, which previews the vanishing gradient in Part 5.

Part 3 [5 points]

Implement the following activation functions in NumPy:

- ReLU
- Sigmoid
- softmax

Please **do not use any other external libraries** for this such as sklearn.

Note: Assume the input `x` is a 1-dimensional numpy array with n entries (i.e. `x.shape` is `(n,)`). The expected output is an array with the same shape.

```
In [17]: def relu(x):
          return np.maximum(x, 0)

          def sigmoid(x):
              return 1.0 / (1.0 + np.exp(-x))

          def softmax(x):
              # shift by the max before exp() so we don't overflow on large inputs
              z = x - np.max(x)
              exps = np.exp(z)
              return exps / np.sum(exps)
```

```
In [18]: # random tests for checking your impementation.
          from tests import tests
          tests.run_test_suite('prob 2.3', (relu, sigmoid, softmax))
```

```
TestRelu (tests.tests.TestActiv.TestRelu) ... ok
TestSigmoid (tests.tests.TestActiv.TestSigmoid) ... ok
TestSoftmax (tests.tests.TestActiv.TestSoftmax) ... ok
```

```
Ran 3 tests in 0.023s
```

```
OK
```

Part 4 [6 points]

Implement the following loss functions in NumPy:

- mean squared error
- mean absolute error
- average hinge loss

Please **do not use any other external libraries** for this such as sklearn.

Note: Assume that the inputs `y` and `yhat` are both 1-dimensional numpy arrays with n entries. The expected output is a float.

```
In [19]: def mean_squared_error(yhat, y):
          diff = yhat - y
          return np.mean(diff ** 2)

          def mean_absolute_error(yhat, y):
              return np.mean(np.abs(yhat - y))

          def hinge_loss(yhat, y):
```

```
# average hinge loss, assuming labels y in {-1, +1}
margins = 1.0 - y * yhat
return np.mean(np.maximum(0.0, margins))
```

```
In [20]: # random tests for checking your impementation.
from tests import tests
tests.run_test_suite('prob 2.4', (mean_squared_error, mean_absolute_error))
```

```
TestMSE (tests.tests.TestLoss.TestMSE) ... ok
TestMAE (tests.tests.TestLoss.TestMAE) ... ok
```

```
-----
Ran 2 tests in 0.028s
```

```
OK
```

Part 5 [6 points]

Explain the vanishing gradient problem. When would you observe this? Give an example.

The vanishing gradient problem is when gradients shrink exponentially as backprop moves toward the early layers, so those layers barely update and learning stalls. It shows up in deep networks with saturating activations like sigmoid or tanh, whose derivative is at most 0.25 — multiplying many such factors together drives the gradient toward zero.

We can already observe it firsthand in our Part 2 calculations: the gradient decreased by over 90% (from 0.0625 at W^3 down to about 0.0039 at W^1) after just two layers. This clearly demonstrates why deep networks with sigmoid activations struggle to train — the error signal simply fades away before it can effectively update the weights of the earliest layers.

[30 Points] Problem 3 - Build a feedforward neural network in NumPy

In this problem you will implement a class representing a general feed-forward neural network that utilizes the sigmoid activation functions. Your tasks will be to implement forward propagation, prediction, backpropagation, and a general train routine to learn the weights in your network via stochastic gradient descent.

The skeleton for the network class is below.

```
In [21]: import argparse
import numpy as np
import pickle
import gzip
import matplotlib.pyplot as plt

class Network:
    def __init__(self, sizes):
```

```

self.L = len(sizes)
self.sizes = sizes
self.biases = [np.random.randn(n, 1) for n in self.sizes[1:]]
self.weights = [np.random.randn(n, m) for (
    m, n) in zip(self.sizes[:-1], self.sizes[1:])]
self.acc_train_array = []
self.acc_test_array = []

def g(self, z):
    """
    activation function
    """
    return sigmoid(z)

def g_prime(self, z):
    """
    derivative of activation function
    """
    return sigmoid_prime(z)

def forward_prop(self, a):
    """
    memory aware forward propagation for testing
    only. back_prop implements its own forward_prop
    should take in weights and biases and return output value
    """
    for W, b in zip(self.weights, self.biases):
        a = self.g(np.dot(W, a) + b)
    return a

def grad_cost(self, a, y):
    """
    gradient of cost function
    Assumes  $C(a, y) = (a - y)^2 / 2$ 
    """
    return (a - y)

def log_train_progress(self, train, test, epoch):
    """ Logs training progres.
    """
    acc_train = self.evaluate(train)
    self.acc_train_array.append(acc_train)
    if test is not None:
        acc_test = self.evaluate(test)
        self.acc_test_array.append(acc_test)
        print("Epoch {:4d}: Train {:.10.5f}, Test {:.10.5f}".format(
            epoch+1, acc_train, acc_test))
    else:
        print("Epoch {:4d}: Train {:.10.5f}".format(
            epoch+1, acc_train))

def SGD_train(self, train, epochs, eta, lam=0.0, verbose=True, test=None):
    """
    SGD for training parameters
    epochs is the number of epocs to run
    eta is the learning rate
    lam is the regularization parameter
    If verbose is set will print progressive accuracy updates
    If test set is provided, routine will print accuracy on test set

```

```

    """
    n_train = len(train)
    for epoch in range(epochs):
        perm = np.random.permutation(n_train)
        for kk in range(n_train):
            self.SGD_step(*train[perm[kk]], eta, lam)
        if verbose and epoch == 0 or (epoch + 1) % 20 == 0:
            self.log_train_progress(train, test, epoch)

def SGD_step(self, x, y, eta, lam):
    """
    TODO: get gradients with x, y and do SGD on weights and biases

    Args:
        x: single sample features.
        y: single sample target.
        eta: learning rate.
        lam: Regularization parameter.

    """
    dW_list, db_list = self.back_prop(x, y)
    # plain SGD update; L2 term lam*W only goes on the weights, not t
    for k in range(len(self.weights)):
        self.weights[k] = self.weights[k] - eta * (dW_list[k] + lam *
            self.biases[k] = self.biases[k] - eta * db_list[k]

def back_prop(self, x, y):
    """
    Backpropagation for derivatives of C wrt parameters
    """
    db_list = [np.zeros(b.shape) for b in self.biases]
    dW_list = [np.zeros(W.shape) for W in self.weights]

    a = x
    a_list = [a]
    z_list = [np.zeros(a.shape)] # Pad with a placeholder so that in

    for W, b in zip(self.weights, self.biases):
        z = np.dot(W, a) + b
        z_list.append(z)
        a = self.g(z)
        a_list.append(a)

    # output layer error, then dC/db and dC/dW for the last layer
    delta = self.grad_cost(a_list[-1], y) * self.g_prime(z_list[-1])
    db_list[-1] = delta
    dW_list[-1] = np.dot(delta, a_list[-2].T)

    # walk backward through the remaining layers
    for ell in range(self.L - 2, 0, -1):
        delta = np.dot(self.weights[ell].T, delta) * self.g_prime(z_list[ell])
        db_list[ell] = delta
        dW_list[ell] = np.dot(delta, a_list[ell].T)

    return (dW_list, db_list)

def evaluate(self, test):
    """
    Evaluate current model on labeled test data

```



```

        """
        ctr = 0
        for x, y in test:
            yhat = self.forward_prop(x)
            ctr += yhat.argmax() == y.argmax()
        return float(ctr) / float(len(test))

def sigmoid(z, threshold=20):
    z = np.clip(z, -threshold, threshold)
    return 1.0 / (1.0 + np.exp(-z))

def sigmoid_prime(z):
    return sigmoid(z) * (1.0 - sigmoid(z))

def mnist_digit_show(flatimage, outname=None):

    import matplotlib.pyplot as plt

    image = np.reshape(flatimage, (-1, 14))

    plt.matshow(image, cmap=plt.cm.binary)
    plt.xticks([])
    plt.yticks([])
    if outname:
        plt.savefig(outname)
    else:
        plt.show()

```

Part 1 [20 points]

Implement `SGD_step`, `back_prop`, and `forward_prop`. Use the following test cases to verify if the code is working as expected.

```
In [22]: from tests import tests
tests.run_test_suite('prob 3', Network)
```

```

TestBackPropWithoutRegularization (tests.tests.TestNetwork.TestBackPropWithoutRegularization) ... ok
TestBackPropWithRegularization (tests.tests.TestNetwork.TestBackPropWithRegularization) ... ok

```

```
-----
Ran 2 tests in 0.089s
```

```
OK
```

Part 2 [10 points]

Run the above Network on MNIST Dataset and report the following (feel free to experiment with different learning rates).

- Change the hidden layer dimensions and experiment with these values: [5, 10, 20].

- Plot accuracies of different hidden layer dimensions vs. epochs for both training and testing.
- Explain the effect of hidden layer dimension on performance.

Note: Accuracies are stored in `self.acc_train_array` and `self.acc_test_array` if `verbose` is `True`.

```
In [23]: location = './data/tinyMNIST.pkl.gz'
f = gzip.open(location, 'rb')
u = pickle._Unpickler(f)
u.encoding = 'latin1'
train, test = u.load()
input_dimensions = len(train[0][0])
output_dimensions = len(train[0][1])
print('Number of Input Features: ', input_dimensions)
print('Number of Output classes: ', output_dimensions)

nns = []
for hidden_layer_dimensions in [5, 10, 20]:
    print('\nHidden Layer Dimensions: ', hidden_layer_dimensions)
    nn = Network([input_dimensions, hidden_layer_dimensions, output_dimensions])
    nns.append(nn)
    nn.SGD_train(train, epochs=200, eta=0.1, lam=0.0001, verbose=True, te
```

Number of Input Features: 196
 Number of Output classes: 10

Hidden Layer Dimensions: 5

Epoch 1:	Train	0.15726,	Test	0.16567
Epoch 20:	Train	0.81753,	Test	0.73349
Epoch 40:	Train	0.87635,	Test	0.79392
Epoch 60:	Train	0.88756,	Test	0.79832
Epoch 80:	Train	0.88956,	Test	0.80152
Epoch 100:	Train	0.89116,	Test	0.79952
Epoch 120:	Train	0.89836,	Test	0.80072
Epoch 140:	Train	0.89476,	Test	0.79472
Epoch 160:	Train	0.89156,	Test	0.80112
Epoch 180:	Train	0.89636,	Test	0.79752
Epoch 200:	Train	0.90036,	Test	0.79952

Hidden Layer Dimensions: 10

Epoch 1:	Train	0.26371,	Test	0.26371
Epoch 20:	Train	0.90996,	Test	0.83954
Epoch 40:	Train	0.93637,	Test	0.85914
Epoch 60:	Train	0.94078,	Test	0.86435
Epoch 80:	Train	0.94638,	Test	0.86234
Epoch 100:	Train	0.94478,	Test	0.86234
Epoch 120:	Train	0.94838,	Test	0.86154
Epoch 140:	Train	0.94838,	Test	0.86795
Epoch 160:	Train	0.95238,	Test	0.86635
Epoch 180:	Train	0.94838,	Test	0.86715
Epoch 200:	Train	0.94718,	Test	0.86515

Hidden Layer Dimensions: 20

Epoch 1:	Train	0.46339,	Test	0.37615
Epoch 20:	Train	0.92037,	Test	0.84834
Epoch 40:	Train	0.94118,	Test	0.86675
Epoch 60:	Train	0.95078,	Test	0.87475
Epoch 80:	Train	0.95598,	Test	0.87115
Epoch 100:	Train	0.95518,	Test	0.87515
Epoch 120:	Train	0.95998,	Test	0.88275
Epoch 140:	Train	0.96158,	Test	0.87835
Epoch 160:	Train	0.96038,	Test	0.88355
Epoch 180:	Train	0.96238,	Test	0.88035
Epoch 200:	Train	0.96359,	Test	0.88275

Plot training and testing accuracies below for each configuration. Use solid lines for testing accuracy, and dotted lines for train accuracy (`ls='dashed'`).

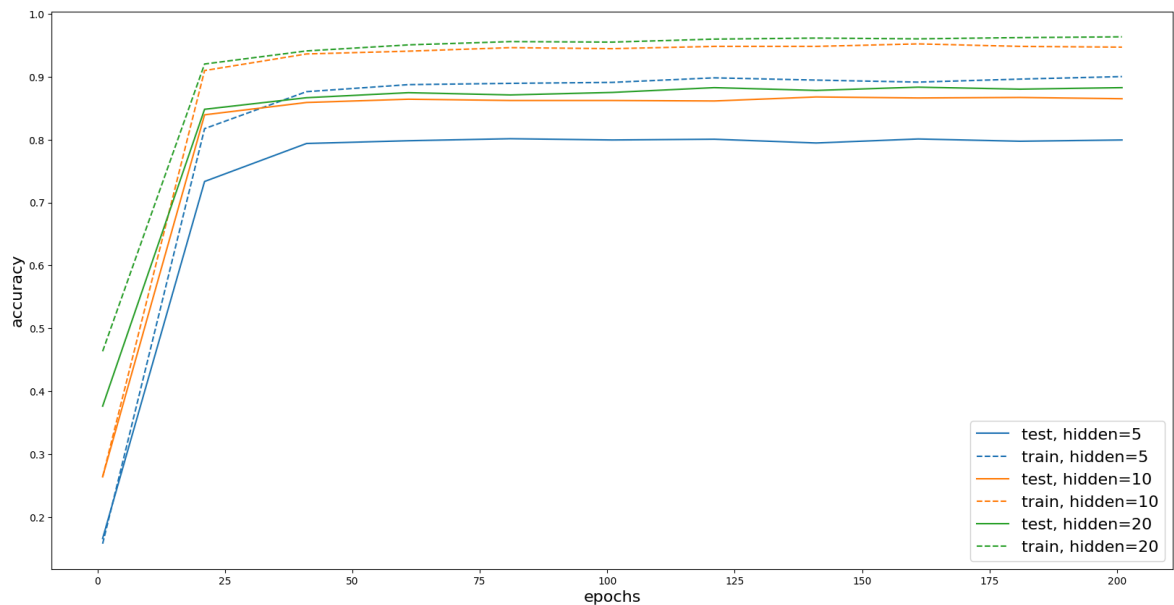
Be sure to label your lines and match colors accordingly.

```
In [24]: fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(20,10))
epochs_array = [i for i in range(1, 220, 20)]

hidden_dims = [5, 10, 20]
colors = ['tab:blue', 'tab:orange', 'tab:green']

for nn, h, c in zip(nns, hidden_dims, colors):
    # solid = test, dashed = train, same color per hidden size
    ax.plot(epochs_array, nn.acc_test_array, color=c, label='test, hidden')
    ax.plot(epochs_array, nn.acc_train_array, color=c, ls='dashed', label='train, hidden')
ax.legend(loc="lower right", fontsize=16)
```

```
ax.set_xlabel("epochs", fontsize=16)
ax.set_ylabel("accuracy", fontsize=16)
plt.show()
```



Give your explanation below.

The hidden layer dimension sets the network's capacity: more units means more weights and a more flexible decision boundary.

Accuracy rises steadily as the hidden size goes from 5 to 10 to 20, on both train and test (test accuracy roughly 0.81, 0.86, 0.88). With only 5 units the network underfits the 196-dimensional inputs and scores lowest everywhere; 10 and 20 units add capacity and lift both curves, with diminishing returns.

The train curve always sits above the test curve, and the gap widens slightly as the hidden size grows, since the extra capacity starts to memorize the training set. Test accuracy still improves at 20 units here, but pushing the size much higher (or dropping the small L2 term) would eventually flatten or hurt test accuracy while train keeps climbing.

[14 points] Problem 4 - Tensors and Autograd

In this problem, we will demonstrate the benefits of PyTorch's Autograd mechanics by converting your implementation from Problem 3 to utilize PyTorch Tensors. This problem is not meant to be difficult, rather it should be quite simple once you've done Problem 3.

Note that there are other ways to go about this. We are keeping things as close to NumPy as possible on purpose. Take a moment to look through our modifications to get an understanding of the differences. In the comments, we'll make notes of native torch functions that perform the same operations.

Here, SGD is implemented manually to show you what's going on behind the scenes, but usually you'd use `torch.optim.SGD`. For reference, the `weight_decay`

parameter implements L2 normalization.

Before we move forward, let's cover a couple of points on notation.

Just as it's best practice to do `import numpy as np`, PyTorch has best practices for imports as well:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

You'll find that many of the functions you've been learning in NumPy have direct translations in PyTorch. The documentation can be found [here](#) and is very user friendly, but we'll cover a few notable differences.

- The numpy parameter `axis` becomes `dim` in PyTorch.
- Tensors have [view](#) operations, which avoid copying data unnecessarily. Use these instead of `reshape()`.
- Using `torch.from_numpy()` also avoids copying data stored in `numpy` arrays.
- Numpy interprets `np.dot()` of 2D matrices as matrix multiplication. Pytorch does not have this behaviour, see instead `torch.matmul()`
- PyTorch uses trailing underscores such as `torch.add_()` to denote in-place operations. As an example, the following operations are equivalent:

```
x = x + y
x = torch.add(x, y)
x.add_(y)
```

You're likely to see these operations when randomly initializing Tensors, or filling a tensor with zeros:

```
torch.nn.init.uniform_(x)
x.zero_()
```

Now onto the problem, your tasks are as follows:

1. Modify `forward_prop` to work on PyTorch Tensors.
2. Modify `SGD_step` to work on PyTorch Tensors. This is easier than it sounds, since with PyTorch we don't have to worry about calculating gradients manually.

In [25]: `import torch`

```
class TorchNetwork(Network):
    def __init__(self, sizes, keep_prob=-1):
        super().__init__(sizes)

        # Convert your weights and biases to Tensors.
        # The flag requires_grad=True tells PyTorch to track gradients for
        self.biases = [torch.tensor(b, requires_grad=True) for b in self.biases]
        self.weights = [torch.tensor(w, requires_grad=True) for w in self.weights]
```

```

def g(self, z):
    """ Activation function
        Could also use torch.sigmoid(z)
    """
    return sigmoid_torch(z)

def forward_prop(self, a):
    """ Memory aware forward propagation.
        Now this is for both training and testing.

    """
    # same as the numpy version, but np.dot -> torch.matmul
    for W, b in zip(self.weights, self.biases):
        a = self.g(torch.matmul(W, a) + b)
    return a

def cost(self, a, y):
    """ Cost function, same as F.mse_loss()
    """
    return ((a - y)**2).mean()

def SGD_step(self, x, y, eta, lam):
    """
        Performs a single step of SGD on weights and biases

    Args:
        x: single sample features.
        y: single sample target.
        eta: learning rate.
        lam: Regularization parameter.

    """
    # Zero out the old gradients if they exist.
    # Would be optim.zero_grad() if we were using an optimizer.
    for p in (*self.weights, *self.biases):
        if p.grad is not None:
            p.grad.detach_()
            p.grad.zero_()

    # forward, loss, then let autograd fill in every .grad for us
    a = self.forward_prop(x)
    loss = self.cost(a, y)
    loss.backward()

    # Perform sgd update
    # Would be optimizer.step()
    with torch.no_grad():
        # l2 regularization on weights only.
        for p in self.weights:
            p.grad.add_(p, alpha=lam)
        # sgd update.
        for p in (*self.weights, *self.biases):
            p.add_(p.grad, alpha=-eta)

def sigmoid_torch(z, threshold=20):
    """ Same as sigmoid before, now with torch.clamp() instead of np.clip
        We could also just use torch.sigmoid().

```

```
.....  
z = torch.clamp(z, -threshold, threshold)  
return 1.0 / (1.0 + torch.exp(-z))
```

```
In [26]: from tests import tests  
tests.run_test_suite('new prob 4', TorchNetwork)  
torch.__version__
```

```
TestBackPropWithoutRegularization (tests.tests.TestNetwork.TestBackPropWith  
houtRegularization) ... ok  
TestBackPropWithRegularization (tests.tests.TestNetwork.TestBackPropWithRe  
gularization) ... ok
```

```
-----  
Ran 2 tests in 0.289s
```

```
OK
```

```
Out[26]: '2.5.1'
```

Acknowledgment : Chris Ketelsen, Chenhao Tan